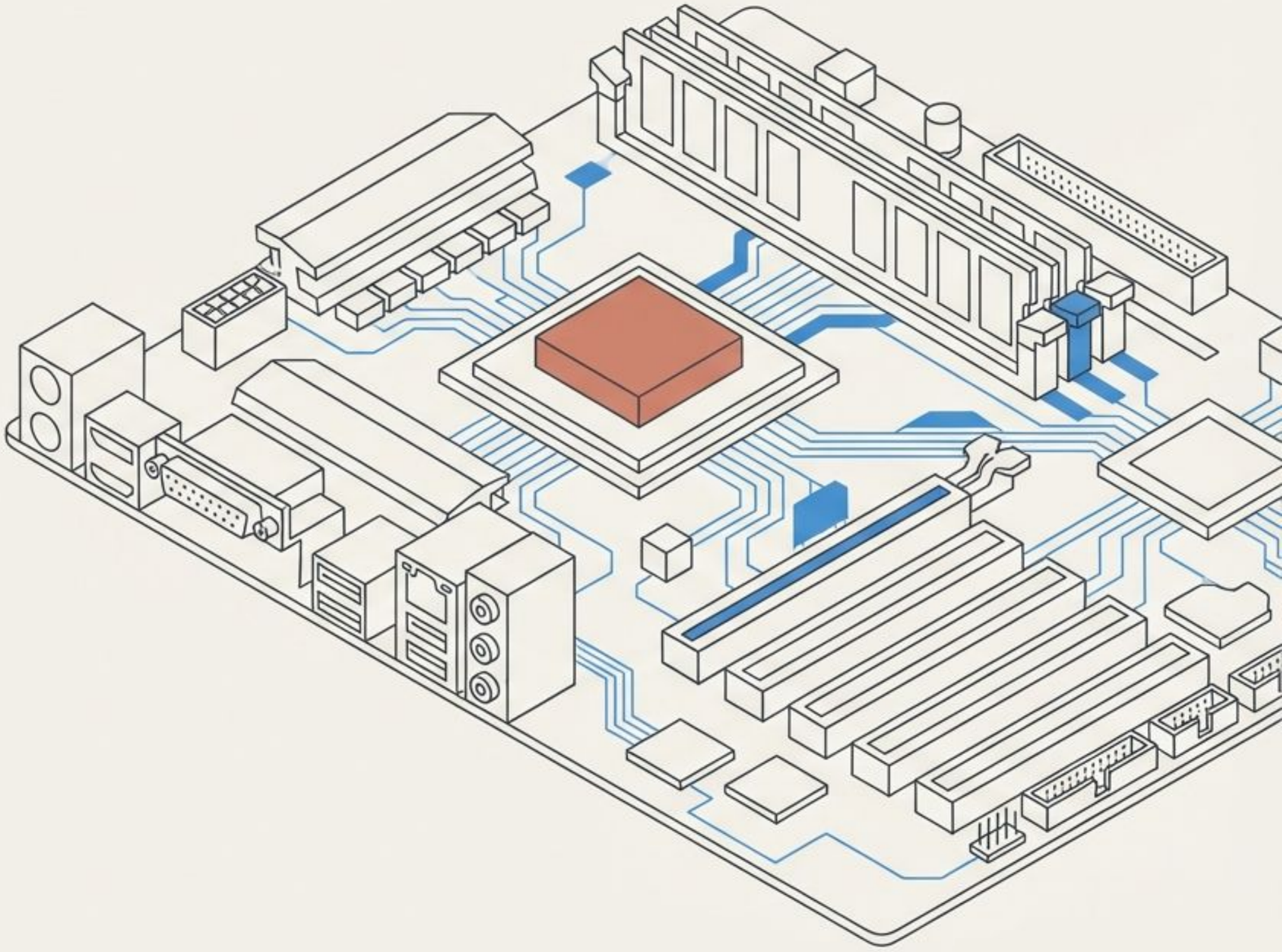
Operating Systems Architecture

The Invisible Conductor of Computing

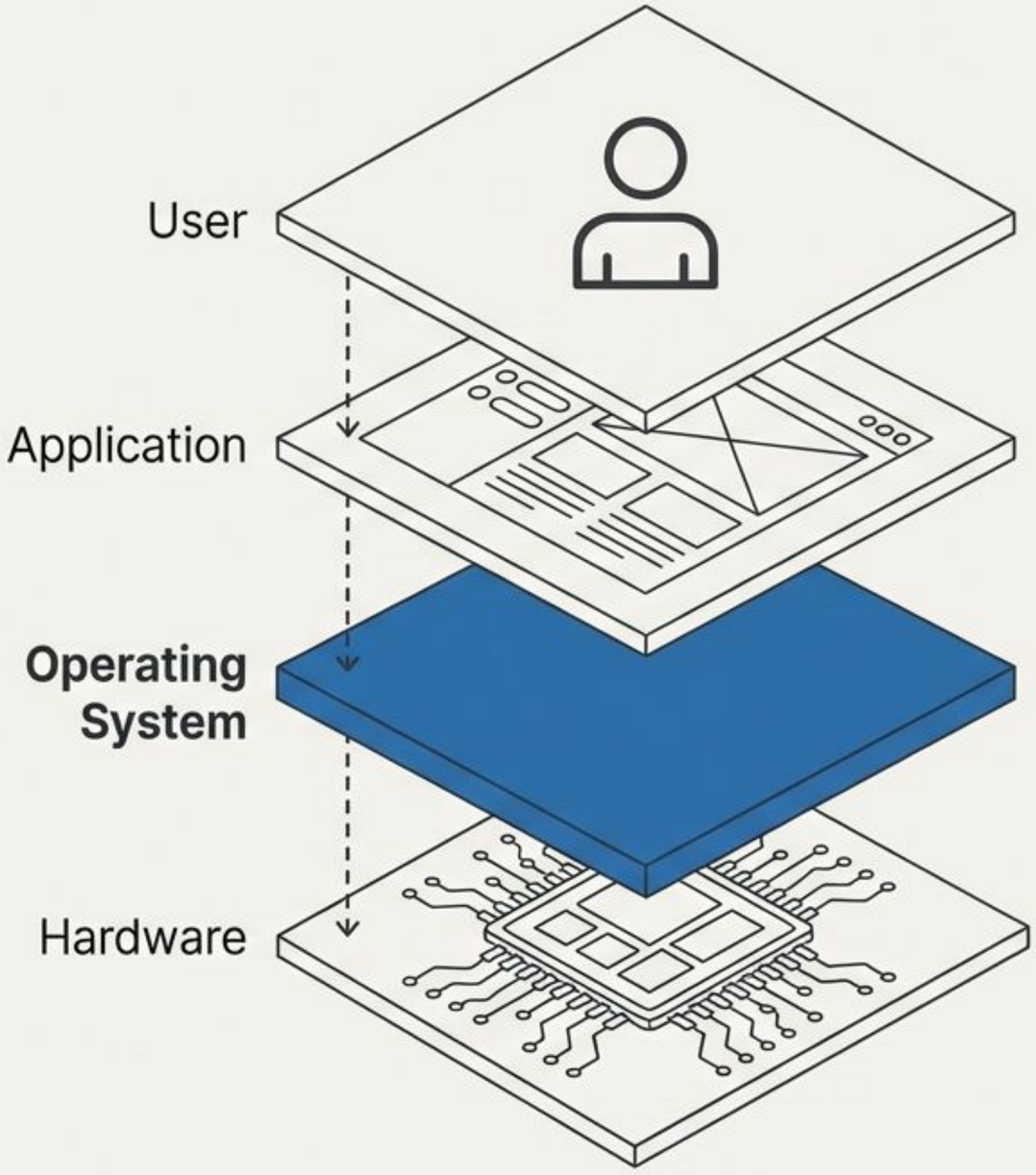
Core Mechanisms, Algorithms, and Real-World Implementation



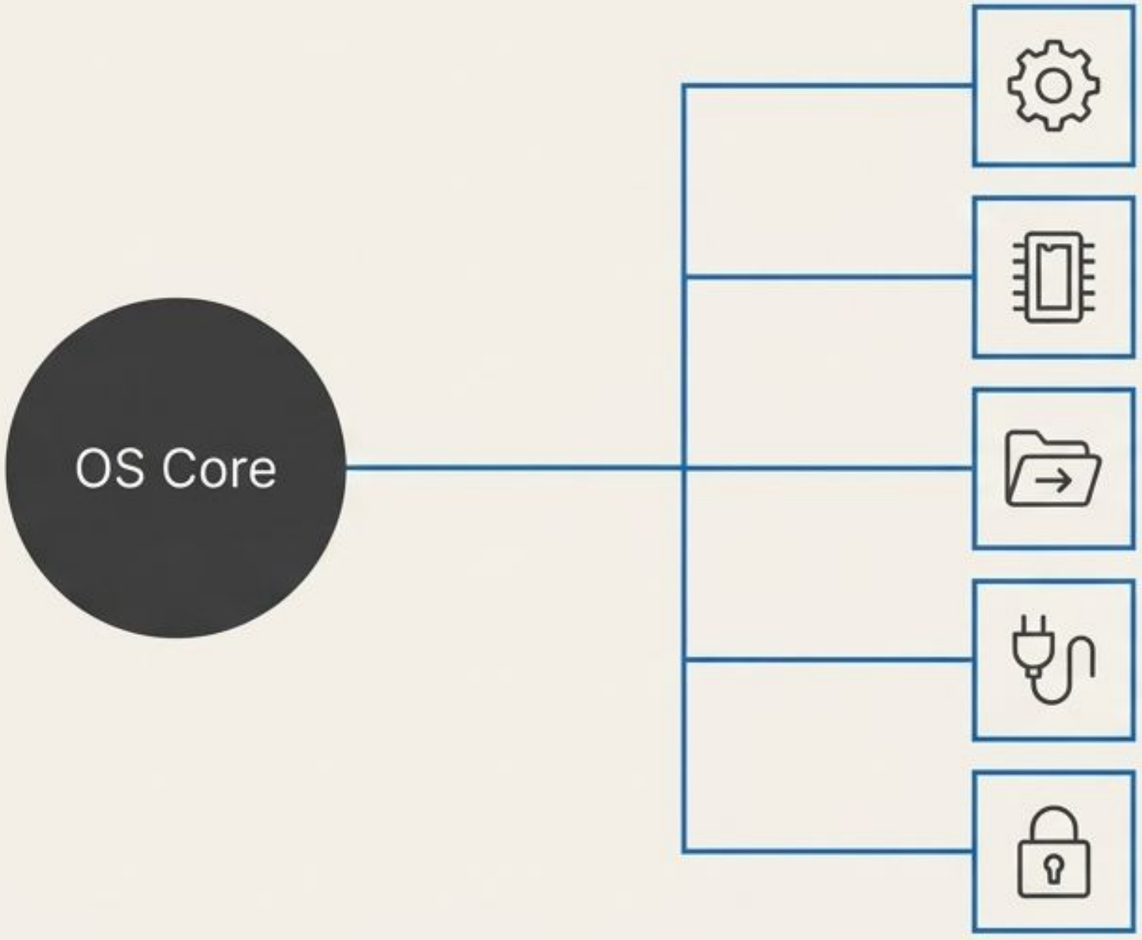
Prepared By : Er. Sharat Maharjan

The Ultimate Intermediary

- **Bridges the Gap:** Connects user applications to raw hardware.
- **Resource Manager:** Allocates CPU, memory, and disk efficiently.
- **Hardware Abstraction:** Hides physical complexities behind clean APIs.
- **User Interface:** Provides GUI and CLI environments.
- **Security & Protection:** Enforces access control and data integrity.



Core Components & Roles



Process Manager: Scheduling & synchronization

Memory Manager: RAM & virtual swapping

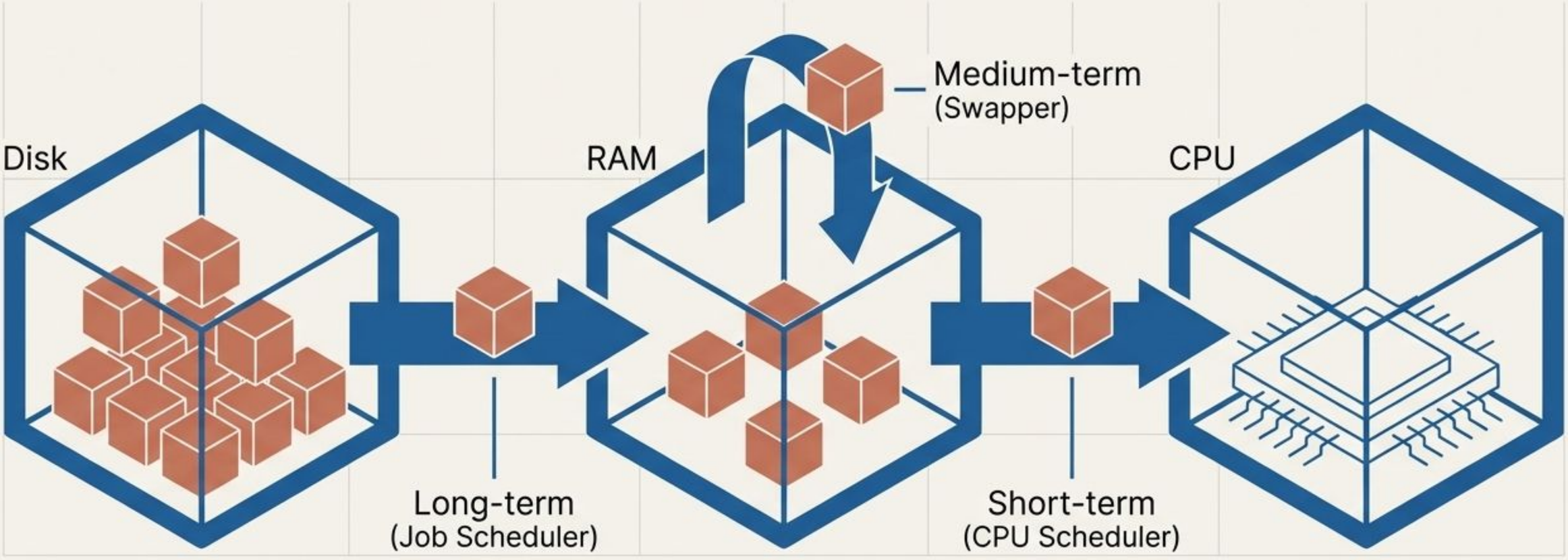
File System Manager: Disks & directories

Device Manager: Hardware drivers

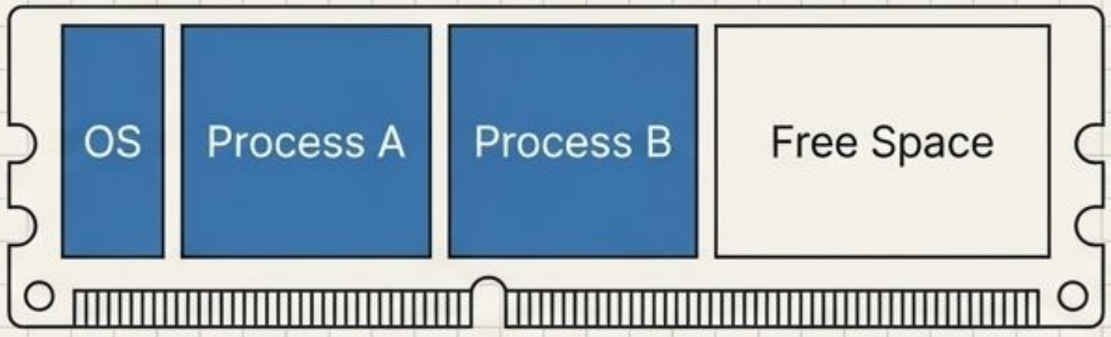
Security Manager: Access control

CPU Scheduling Mechanisms

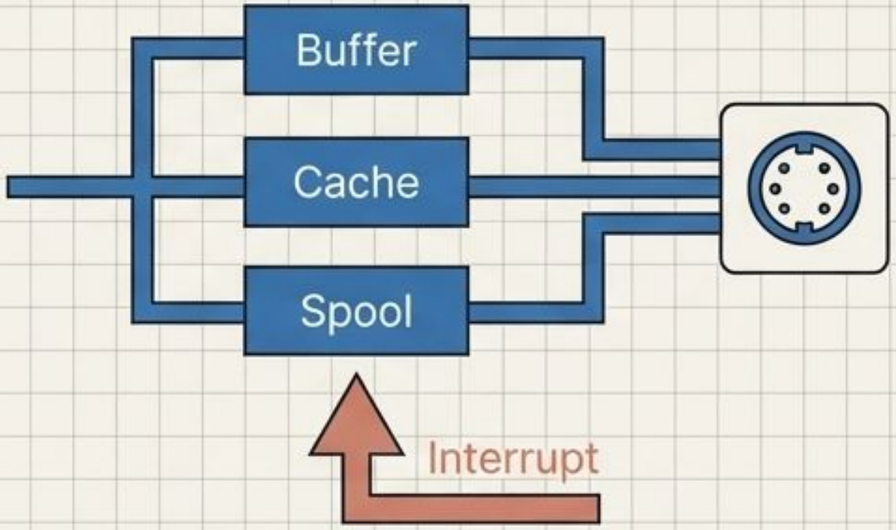
Goal: Decide which process gets access to the CPU and when.



Managing Memory & I/O



- Memory Allocation: Grants and revokes RAM space.
- Virtual Memory: Expands capacity via Paging and Segmentation.



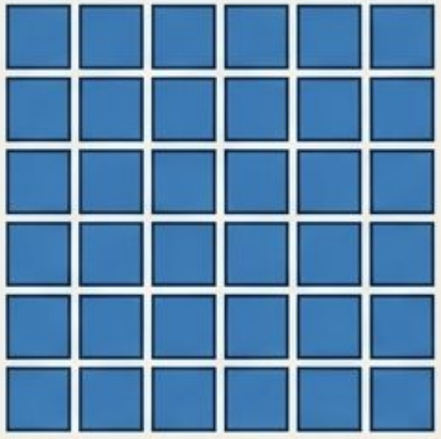
- I/O Drivers: Translates commands into hardware actions.
- I/O Optimization: Uses buffering, caching, and spooling.
- Interrupt Handling: Manages asynchronous signals.

Analysis: Scheduling Trade-offs

Algorithm	Advantage	Disadvantage	Efficiency
FCFS	Simple, no starvation	Poor for short jobs (Convoy Effect) ⚠	
SJF	Optimal minimum wait time	Hard to predict burst time	
Round Robin	Fair, good response time	High overhead with small quantum	
Priority	Important processes run first	Starvation for low-priority ⚠	

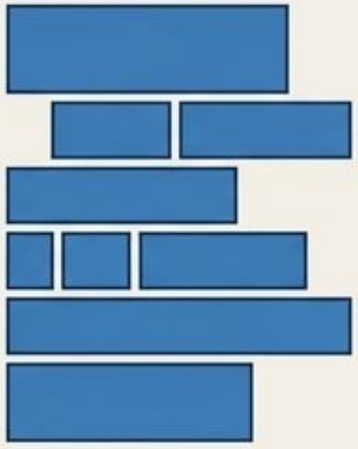
Analysis: Memory Techniques

Paging



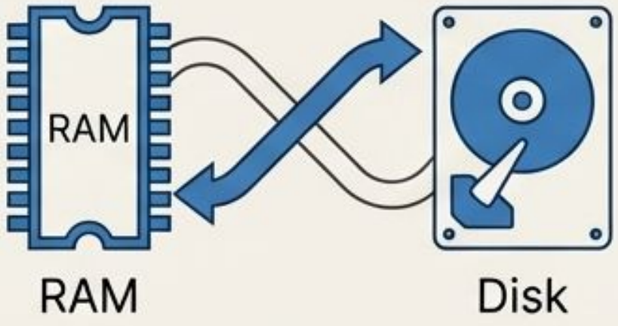
+ No external fragmentation
− Internal fragmentation and table overhead

Segmentation



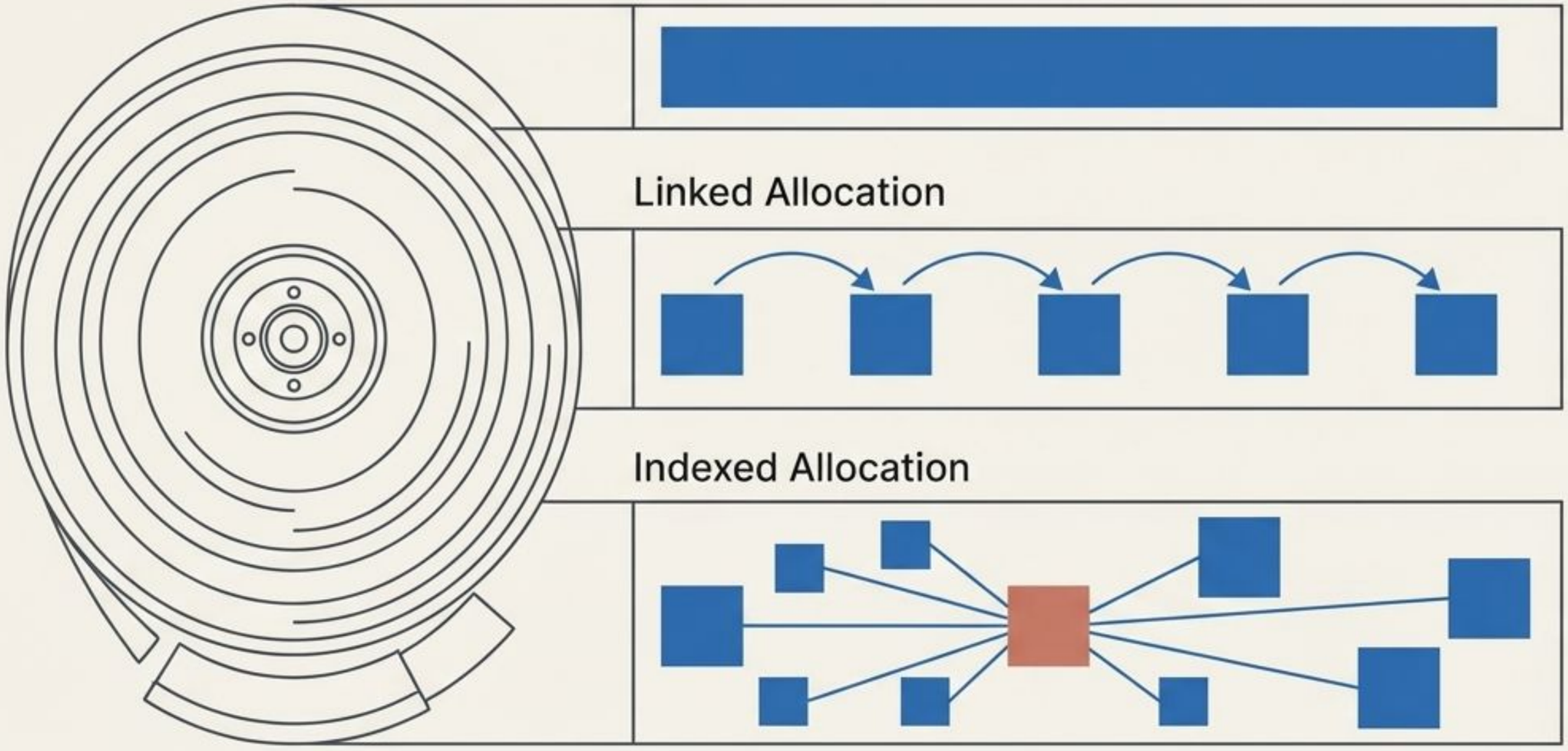
+ Logical division, easy sharing
− Risks external fragmentation

Virtual Memory



+ Runs programs larger than RAM
− Thrashing if excessive paging occurs

Analysis: File System Storage



Fast sequential access but suffers from severe fragmentation.

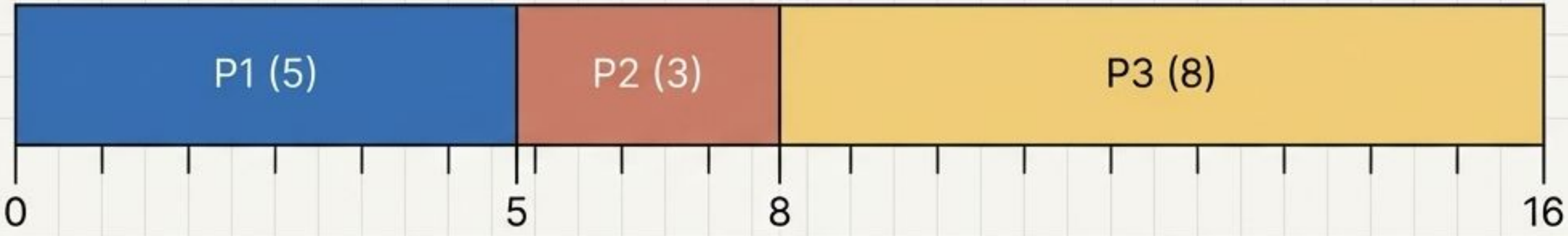
Eliminates fragmentation but random access is painfully slow.

Supports excellent random access but requires index block overhead.

Algorithm Simulation: Time & CPU

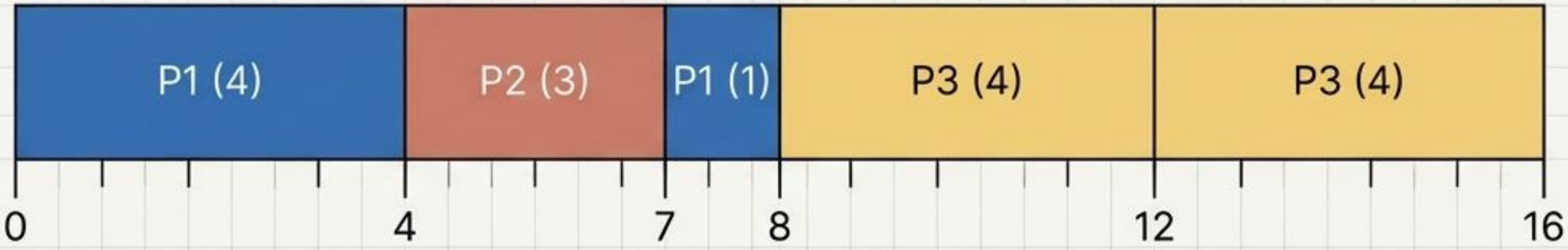
Scenario Processes: P1 (Burst=5), P2 (Burst=3), P3 (Burst=8)

FCFS Wait Time:



P1 stalls faster jobs (Convoy Effect). Avg wait = 4.33ms.

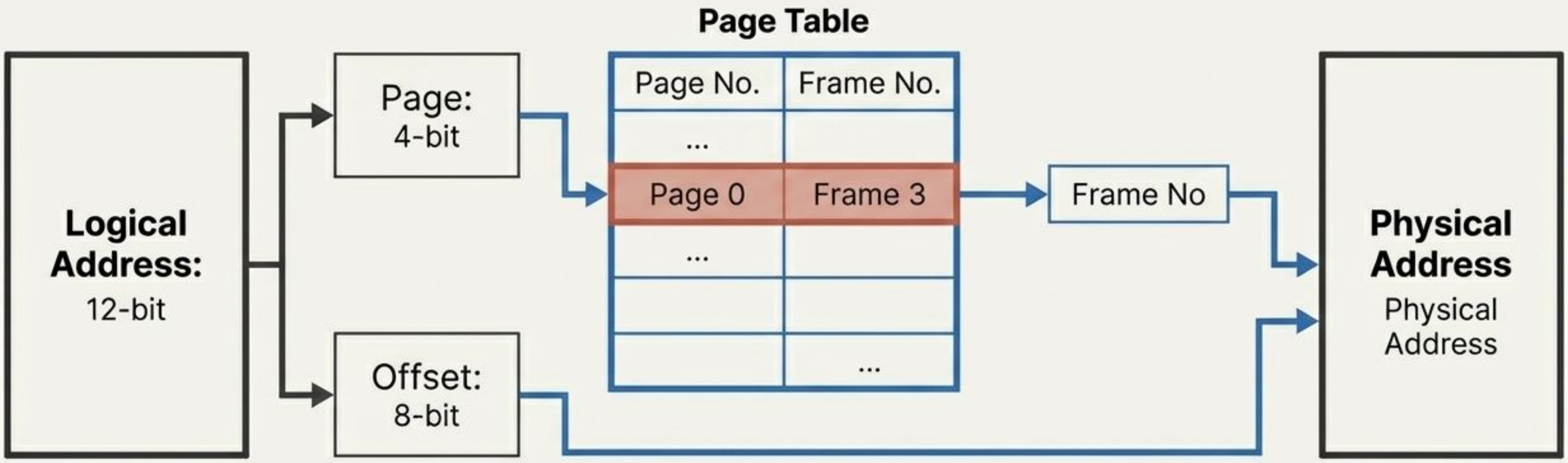
Round Robin (q=4):



Interleaved execution ensures fairness, despite higher overhead.

Algorithm Simulation: Memory Paging

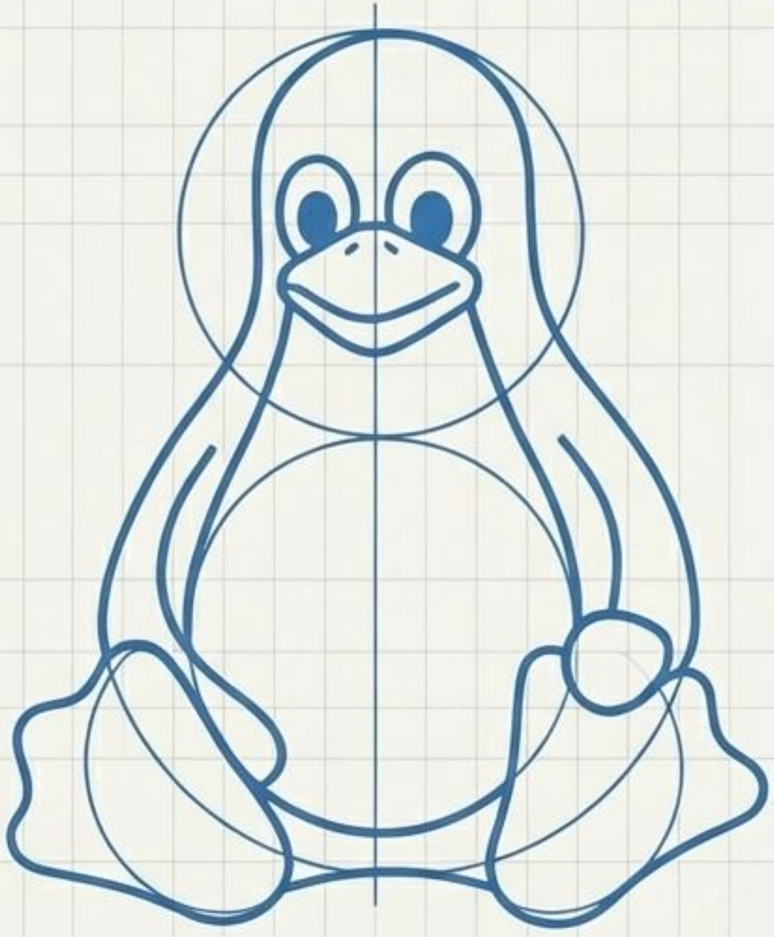
Goal: Convert Logical Address to Physical Address.



Physical Address = (Frame No x Frame Size) + Offset

Real-World Masterclass: Linux Architecture

Open-source, highly modular kernel architecture.
Enterprise-grade resource management.



[CFS]

CPU: Completely Fair Scheduler (CFS)

[BUDDY SYSTEM]

Memory: Buddy System & Slab Allocators

[EXT4]

File/IO: Ext4 and specialized I/O schedulers

Linux Under the Hood: CPU & RAM

Completely Fair Scheduler (CFS): Uses a Red-Black Tree for O(log n) task selection.

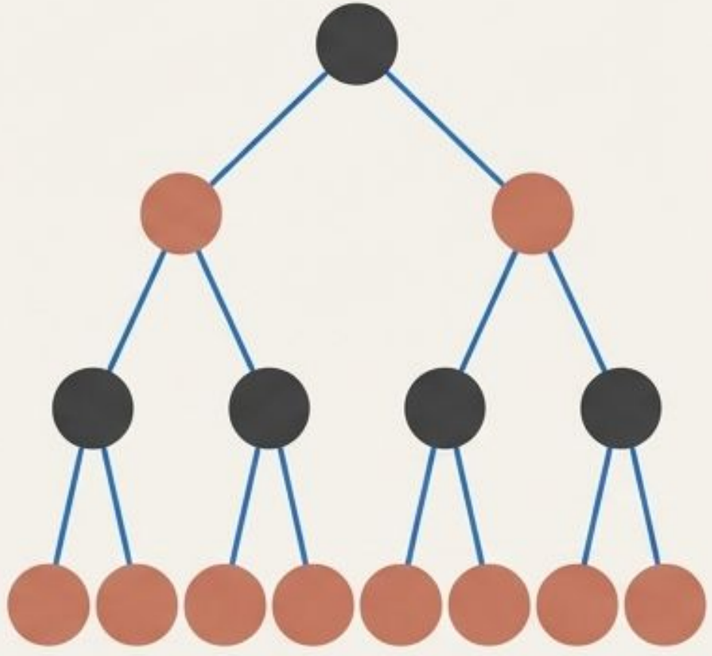
`vruntime`: Tracks execution time to ensure perfect fairness.

Buddy System: Allocates contiguous memory by halving blocks.

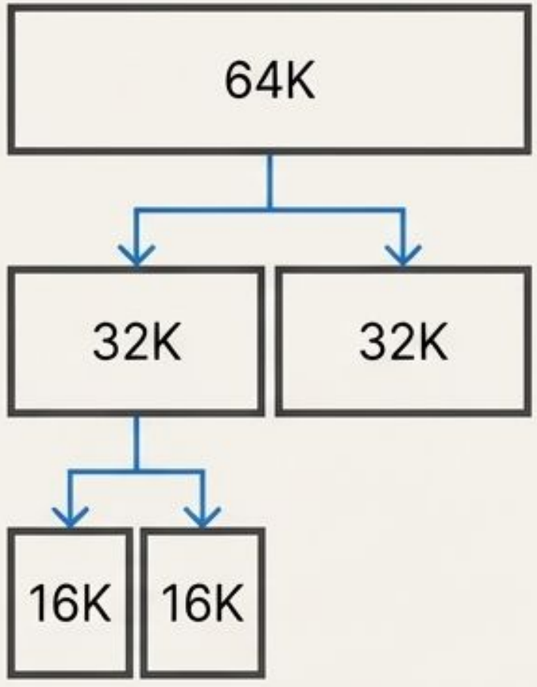
Slab Allocator: Caches frequently used kernel objects.

`swappiness`: Parameter controlling disk-swapping tendency.

CFS
(Red-Black Tree)

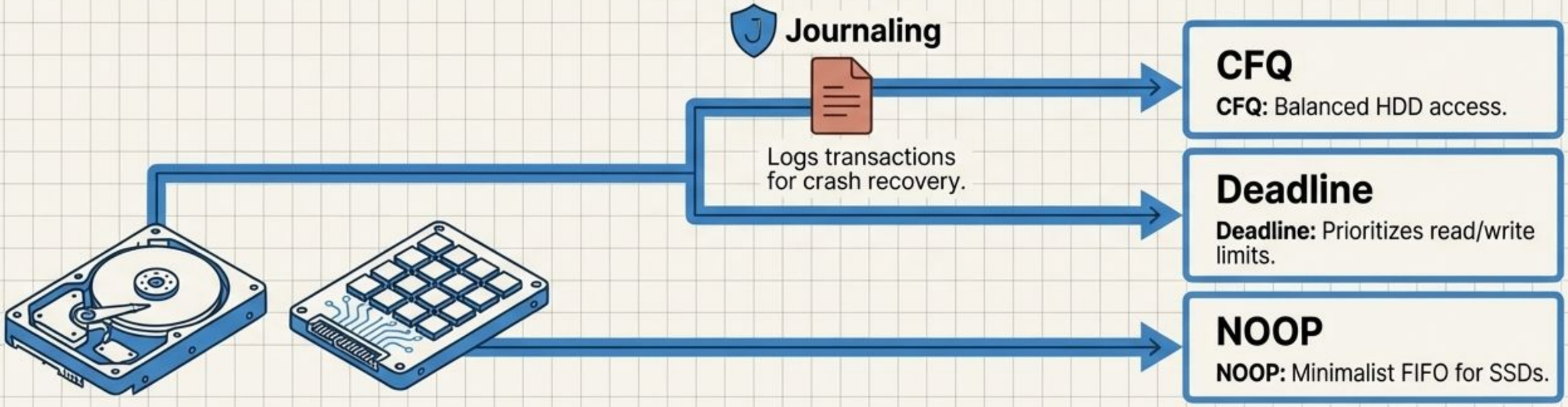


Buddy System
(Cascading Blocks)



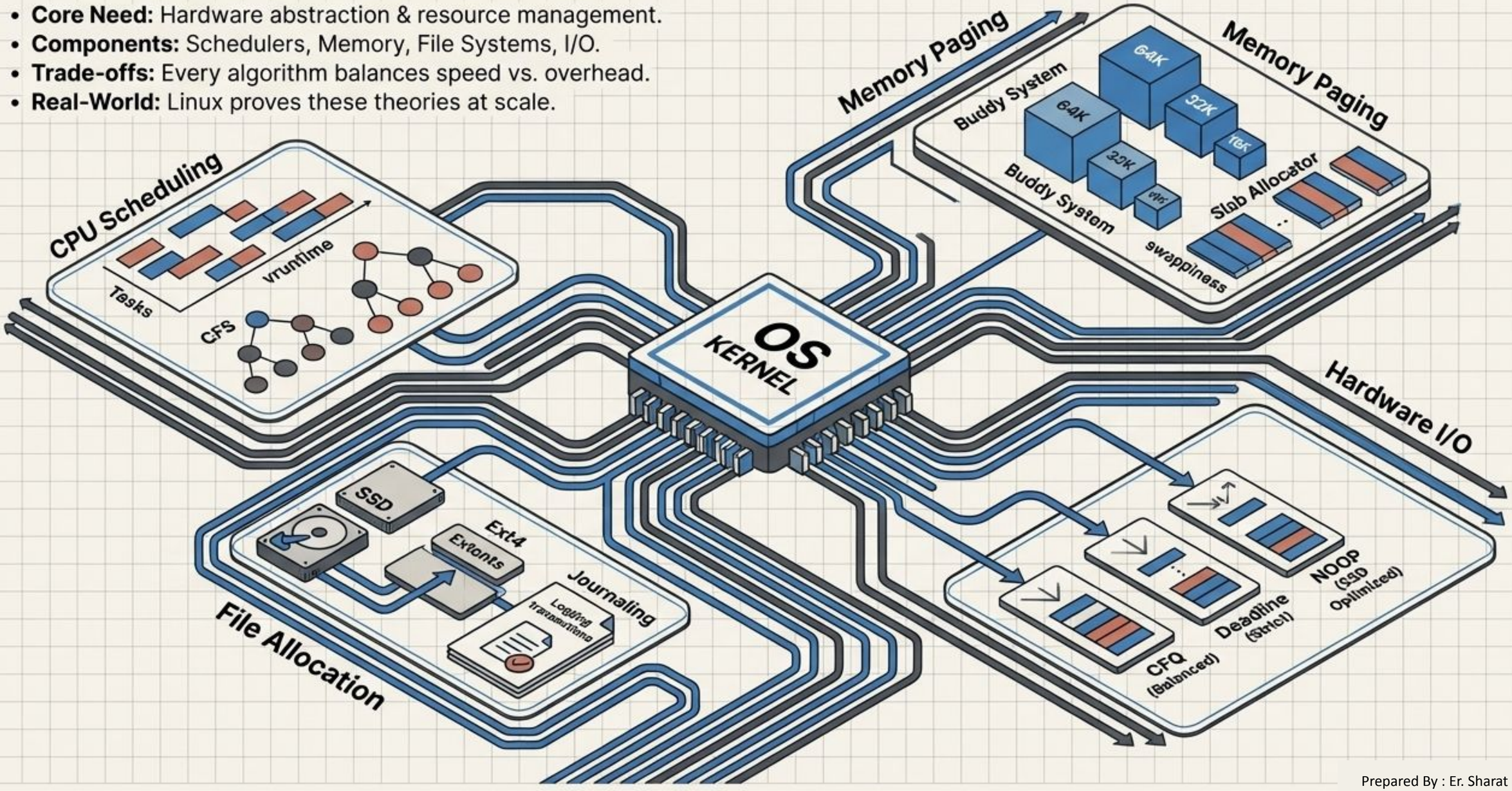
Linux Under the Hood: Disk & Data

Ext4 File System: Uses Extents to heavily reduce fragmentation.
Journaling: Logs transactions for crash recovery and integrity.
CFQ: Completely Fair Queuing for balanced HDD access.
Deadline: Prioritizes strict read/write time limits.
NOOP: Minimalist FIFO scheduler, explicitly optimized for SSDs.



The Complete OS Framework

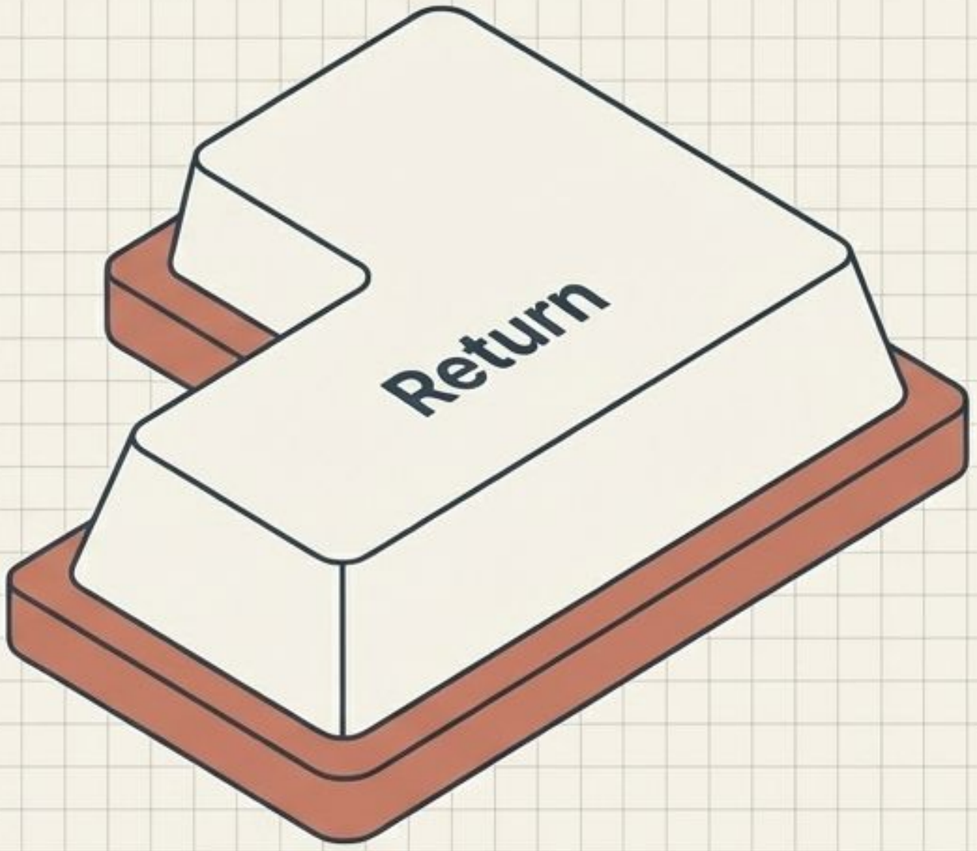
- **Core Need:** Hardware abstraction & resource management.
- **Components:** Schedulers, Memory, File Systems, I/O.
- **Trade-offs:** Every algorithm balances speed vs. overhead.
- **Real-World:** Linux proves these theories at scale.



Module Synthesis & Exam Focus

Analyze: Algorithm pros & cons (FCFS vs RR, Paging vs Segmentation, Ext4).

Calculate: Practice FCFS / Round Robin math & logical-to-physical address translation.

Apply: Map theoretical managers to Linux-specific implementations (CFS, Buddy System, NOOP).

Synthesize: Explain the OS as the ultimate resource intermediary.